

Conceptual practice: Lectures 1–5

Including Practice Notebooks 1 and 2

Questions only. No numerical evaluation. Solutions are in a separate file.

How to use this

Each question asks for a reason, a comparison, or a qualitative picture. A formula is a name for an object. If the urge is to compute a number, the question is about something else: what that object is doing.

Two questions share each idea. Attempt the first, then read only that solution, close it, and answer the second out loud in three or four sentences before writing it down. A second question that you can answer only by remembering the first solution's wording is not yet understood.

1 What the framework is for

Idea

The Neural Engineering Framework is a way to turn a vector-valued description, together with biological constraints such as tuning, neuron count, and time constants, into a spiking network. The population, not the single neuron, is the unit that represents a value. The Semantic Pointer Architecture is a later tool for building symbol-like objects out of vectors. It is not required to represent or transform a vector.

1. A classmate says the framework is a theory of how synapses learn. What job are the “optimal weights” actually doing, and what kind of description goes in the front of that translation?
2. Why does a single neuron's spike train fail as the definition of “the represented value,” even when that neuron is tuned to the stimulus?

2 Representation, transformation, dynamics

Idea

Representation maps a low-dimensional value to population activity and back. Transformation chooses connection weights so that a downstream population represents $f(x)$. Dynamics uses recurrence so the value obeys a differential equation. Feedforward weights do not implement dynamics. Backpropagation can change what every layer represents because it does not require that intermediate representation to be chosen in advance.

1. Assign each of the three principles to one of these jobs: storing a vector in activity, computing a function of that vector, and making the vector evolve in time. Which job needs a connection that returns to the same population?
2. You know the quantity each population should represent and the function on each connection. Why can the weights be solved one connection at a time? What becomes unavailable if you do not know those intermediate values?

3 What a LIF neuron is doing

Idea

Between spikes, voltage is pulled toward the current J and leaks back toward rest. Crossing threshold produces a spike, a reset, and a refractory hold during which another spike cannot be forced. If J is at or below threshold, the pull never arrives in finite time. The rate formula $G(J)$ throws away the spike times and keeps only how often the threshold is hit.

1. In one sentence each, say what the leak, the threshold, and the refractory period each prevent or produce.
2. Voltage climbs toward threshold and flattens just under it. What is true of J ? Why does running the neuron for longer not eventually produce a spike?

4 Rate curves, and why they bend

Idea

A LIF rate curve is silent at and below threshold, steep just above it, and bends over because the refractory period caps the rate at $1/\tau_{\text{ref}}$. A ReLU is a straight line through the origin once it turns on. A purely linear map $G(J) = J$ cannot build a new function by stacking layers. A tanh rate goes negative, which is not a firing rate.

1. Compare a LIF rate curve and a ReLU just above threshold, and again at very large J . Where do they agree in spirit, and where does the LIF stop behaving like a straight line?
2. Why does the course treat nonlinearity as necessary for computation? Why is tanh a poor stand-in for a firing rate even though it is nonlinear?

5 Response curves and tuning curves

Idea

A response curve plots rate against current J . A tuning curve plots rate against the represented stimulus. The encoding equation turns the stimulus into a current, then passes that current through the response curve. Positive correlation, negative correlation, and a preferred stimulus are three shapes seen in data. In one dimension the sign of the encoder flips the slope. A preferred stimulus appears when the stimulus is a vector and the current depends on alignment with an encoder.

1. You are handed a plot of firing rate against sound intensity, and another of firing rate against injected current. Which is a tuning curve, which is a response curve, and what object converts one horizontal axis into the other?
2. A neuron fires less as a scalar stimulus grows. A second neuron fires most at one particular stimulus value and falls off on either side. Which encoding ingredient produces each shape?

6 Encoders, intercepts, and the radius

Idea

The encoder is a unit vector: the stimulus direction that drives the neuron hardest. The neuron turns on when the projection $\langle e, x \rangle$ passes the intercept. In one dimension, $e = +1$ fires on the high side of the intercept and $e = -1$ fires on the low side. The population is built for a ball of radius r . Outside that ball the tuning and the decoders were not set.

1. Two one-dimensional neurons share an intercept $\xi > 0$. One has $e = +1$ and one has $e = -1$. For each, describe the side of the number line on which it fires.
2. A represented value spends part of its time outside the ball the population was built for. What kind of failure should you expect, and why is “the output hard-clips at the radius” too strong a claim?

7 A linear readout of a nonlinear code

Idea

Encoding is nonlinear. Decoding estimates the value by a weighted sum of activities. The code is distributed, so several neurons carry the value together, and it is lossy, because a finite set of curved tuning curves cannot in general be combined into a perfect copy of x . A special pair of opposing ReLUs can reconstruct a scalar exactly. Random LIF curves cannot.

1. Why is a distributed code more tolerant of a few noisy neurons than a code that stores the whole value in one neuron? Why does that tolerance still leave the representation lossy?
2. Two ReLU neurons with opposite encoders, unit gain, and zero bias can be given decoders $(1, -1)$ and reconstruct x exactly. Random LIF tuning curves, decoded linearly, cannot. What property of the LIF curves is the obstacle?

8 Noise, and why decoders shrink

Idea

Neural activity is noisy: spike counts in a short window, vesicle release, channel openings, and network irregularity. Principle 0 says an analysis that ignores this is incomplete. Unregularized decoders can become large, and large weights pass activity noise straight into the estimate. The penalty $N\sigma^2 I$ is added while fitting on clean activities. It shrinks the weights. Clean reconstructions get slightly worse. Noisy reconstructions get much better. The same penalty also keeps AA^T invertible when two neurons are nearly copies of each other.

1. Decoders are fitted on noiseless tuning curves, then used on activities with noise added. The estimate swings wildly. Why do large decoding weights cause that, and what does the regularizer do to those weights?
2. Name one biological reason to include noise, and one purely mathematical reason the same penalty term is welcome even before you care about biology.

9 Distortion and noise are different errors

Idea

Static distortion is the leftover mismatch because a linear decoder is combining curved tuning curves. Noise error is the part that grows when decoder weights are large. With more neurons, squared noise error falls like $1/n$ and squared distortion like $1/n^2$. In the noise-limited regime the root-mean-square error therefore falls like $1/\sqrt{n}$. Shrinking σ by ten, with the decoders held fixed, shrinks the squared noise error by a hundred.

1. Which error would remain if every neuron were a perfect, noiseless implementation of its tuning curve? Which error would remain if the tuning curves could form x exactly but the activities were still noisy?
2. You double the population. Both errors fall. Which one falls faster, and why does the root-mean-square error in a noise-dominated population fall more slowly than the squared noise error?

10 Vectors, preferred directions, and the population vector

Idea

In two dimensions the current depends on $\langle e, x \rangle$. Threshold is a line perpendicular to the encoder. On the unit circle the projection is a cosine of the angle between x and e , and the rate is that cosine after the response curve, so it peaks at one angle. Using the encoders themselves as decoders points roughly the right way for a broad, uniform population. The lengths of those decoders are wrong. A fitted decoder fixes the lengths.

1. A two-dimensional neuron has encoder e and intercept ξ . Describe the set of stimuli where it is silent, and the direction in which its rate grows fastest.
2. You decode a two-dimensional value twice: once with a fitted decoder, once using the encoder matrix in place of the decoder. What should look similar, and what should look systematically wrong?

11 Spikes are not rates

Idea

A rate $G(J)$ is a summary. A spike train is a sum of deltas, so it is zero between spikes. A rate-mode decoder applied to raw spikes therefore has nothing to read most of the time. A filter spreads each spike over time and rebuilds a rate-like trace. A discrete spike stored as the number 1 does not have the area of a delta. The factor $1/\Delta t$ has to appear on the spike, on the kernel, or on the decoded output, in one place.

1. Immediately after a spike, and long before the next one, a decoder that multiplies the raw spike train by d reports zero. What fact about the spike train causes that, and what is the filter restoring that the rate model never had to restore?
2. A simulator stores a 1 in the time bin where a spike happened. The rate decoder was fitted in hertz. Why is that stored 1 the wrong height, and why does inserting $1/\Delta t$ twice make the result wrong in the other direction?

12 Smooth signals and their spectra

Idea

White noise drawn independently at every time step is as jagged as the time step allows, and its spectrum is broad. Band-limited white noise zeros the Fourier coefficients past a limit, so the time signal cannot change faster than that limit. A real signal needs conjugate-symmetric coefficients, which puts power at both $+\omega$ and $-\omega$. The ensemble-average magnitude is flat inside the band because every in-band coefficient was drawn from the same distribution. One draw looks ragged. Holding the RMS fixed and widening the band keeps the typical size of the time signal and makes the wiggles faster. The same power is then shared across more frequencies, so the average spectral plateau gets lower.

1. You generate noise by drawing an independent Gaussian at every time step. Why is that a poor model of a physical stimulus, and what does zeroing the high-frequency coefficients change in the time trace?
2. Two signals have the same RMS. One is band-limited to a low frequency and one to a high frequency. What matches in the time domain, what does not, and what happens to the height of the average $|X|$ plateau when the band gets wider?

13 Why the optimal filter is a low-pass

Idea

At each frequency the best linear filter multiplies the response by $H(\omega)$. The numerator is the average product of the input with the conjugated response. Where the input has no power, and the leftover response is uncorrelated with it, that product averages to zero, so H is zero there. Spikes are sharp, so the response spectrum continues past the input band. The decoded output is the response times H , so it is confined to the input's frequencies. The resulting $h(t)$ looks into the future. A synapse cannot.

1. The spike train has power far above the input's cutoff. The optimally decoded output does not. What is present in the response at those high frequencies, and what is absent from the numerator of H ?
2. Calling the optimal filter “acausal” is a statement about $h(t)$. What does the filter need that has not happened yet, when is that legitimate, and which biological object replaces it when the decode has to run online?

14 Synaptic filters

Idea

A postsynaptic kernel is causal: zero before the spike. The integer n is the power of t in front of the exponential, not a frequency. For $n = 0$ the kernel jumps at the spike and decays. Larger n delays the peak. Larger τ spreads the same unit area over a longer time, which suppresses rapid jitter and also lags and attenuates fast parts of the signal. A slow input and a fast input of the same RMS, through one fixed filter, differ in how much of the signal itself is distorted. Both still contain leftover spike noise.

1. What does n count? Describe the moment of the spike for $n = 0$ and for $n = 1$, and say which peak comes first.
2. Increasing τ changes a decoded trace in two ways. Separate those two ways. Then say why a 2 Hz input is usually less distorted than a 10 Hz input through the same filter, without claiming that the slow input has lost its spike noise.

15 Weights are the decoder and the encoder folded together

Idea

A communication channel decodes a value from one population and encodes it in the next. Both steps that touch the value are linear, so they collapse into one matrix $W = ED$. Biology has synapses, not a decoder sitting between populations. Simulators still decode and re-encode because the represented dimension is small and the full weight matrix is large. Dale's principle says one presynaptic neuron is either excitatory or inhibitory. A matrix built as ED usually gives that neuron both signs.

1. Start from “decode, then encode.” Why is there a single weight from each presynaptic neuron to each postsynaptic neuron, and why is a box labelled “decoder” not a biological object?
2. The connection already works if you store W . Why do simulators avoid building that matrix, and which biological signing rule does a stored $W = ED$ usually break?

16 Decoding a function, not the identity

Idea

A function decoder is the same least-squares problem with the target replaced by $f(x)$. The decoder is linear in that target, so a sum of targets uses the sum of decoders. Doubling an identity decoder produces $2x$ and misses an added constant. Tuning curves are smooth, so smooth targets are easier to match than targets with jumps. The error still falls as the population grows.

1. You have an identity decoder and you want the connection to compute $2x + 1$. Why does doubling the identity decoder fail, and what second decoder supplies the missing piece?
2. The same population decodes a smooth curve and a curve with a jump. Which reconstruction is closer to its target, and what property of the tuning curves is responsible?

17 More than one incoming population

Idea

Currents from separate populations add. Two connections therefore compute a function of x alone plus a function of y alone. A product mixes x and y inside one function, so both values have to be represented together before the function decoder runs. A nonlinearity of one variable, such as y^2 , does not need a joint population. Expanding $(x + y)^2$ produces a cross term, so adding a decoded x^2 to a decoded y^2 is short of the square of the sum. Summing first and then squaring the scalar sum does compute it.

1. External x and y arrive in separate scalar populations. Why is $3y - x$ a legal pair of connections, and why is xy not?
2. A network adds a decoded x^2 to a decoded y^2 and hopes to represent $(x + y)^2$. What term is missing? Describe a network that computes the square of the sum without building a two-dimensional population.

18 What a chain adds, even when the decoder is right

Idea

Each synaptic stage delays the value by about one time constant. A filter that starts at zero also needs time to charge at the beginning of a trial. Spikes leave ripple on the decoded trace. Driving the population past its radius is a different failure: the decoders were never asked to work there. Lag, startup, and ripple can all appear while the value stays inside the radius.

1. A value passes through three feedforward populations, each synapse using the same τ . The decoders are excellent. What should you still expect about the timing of the output?
2. During the first few tens of milliseconds the decoded trace sits near zero, then catches up and looks jagged. The represented value never leaves the training radius. Which two mechanisms is that, and which mechanism have you ruled out?

When the second question in every section can be answered without opening the solution, the original practice test is the same material with the idea boxes removed. On that test, name the mechanism before drawing.